Concordia University — Gina Cody School of Engineering and Computer Science

SOEN 321

Information Systems Security

Winter 2026

Option 2: Design and Analyze a Secure Communication System

Project Title: *A Multi-Algorithm Encrypted File Sharing System with Cryptographic Integrity Verification*

Instructor: Dr. Ayda Basyouni

Team Name: Git Going

Due Date: 2026-04-14

Name	ID	Contribution
Yassine Ibhir	40251116	Encryption & hashing
Tanim Chowdhury	40245607	Security Analysis and Discussion
Miskat Mahmud	40250110	Report: Introduction, System Design, Scenario, Cryptographic technique, Zero-knowledge server model Coding: Register and login, logout routes in auth.py US 1 account setup
Omar Elmasaoudi	40255123	- Upload: encrypts files with a random FEK wrapped by the user's RSA public key; server never stores plaintext - Download: re-derives private key via PBKDF2, unwraps FEK, decrypts file, and verifies integrity - List: GET /files returns all files owned by the authenticated user - Registered files blueprint in app.py
Adib Akkari	40216815	System Design, Requirements, DB schema, Sequence Diagrams
Shaheer Mohammad	40252466	Backend, File sharing, deletion & edit routes, Implementation, Security Analysis
Ziad-Tarik Taufeek	40205732	Worked on the Abstract, Conclusion and validation of diagrams with Adib & requirement Analysis

Abstract

This paper presents the design and implementation of a hybrid-encrypted secure file sharing platform built to demonstrate applied cryptographic principles in a web-based system. The platform implements a layered cryptographic stack: files are encrypted using AES-256-GCM or ChaCha20-Poly1305 with a per-file random File Encryption Key (FEK), which is itself wrapped using RSA-2048 with OAEP padding so that only the intended recipient can recover it. User passwords are hashed with bcrypt at cost factor 12, and each user's RSA private key is encrypted at rest using a key derived via PBKDF2-HMAC-SHA256 with 100,000 iterations, ensuring that a full database breach yields no immediately usable credentials or file contents. File integrity is verified by hashing plaintext before encryption and recomputing the hash after decryption, with support for SHA-256, SHA-512, and MD5 the last included deliberately as a broken algorithm for comparison. Security analysis reveals that while the architecture successfully eliminates plaintext and raw key storage at rest, the server-side placement of all cryptographic operations means the server processes plaintext transiently in memory, which represents the primary gap between this prototype and a true zero-knowledge design.

1. Introduction

Individuals and organizations upload sensitive files to cloud platforms all the time. They trust that their data will remain private. But data breaches have become common nowadays. For example: exposed medical records, leaked corporate documents, financial data etc. . Many of these incidents trace back to a simple architectural flaw: the server holds the keys. If a storage provider controls encryption, data breaches can occur in many ways. Such as: a single compromised server, a rogue employee, or a government subpoena. The problem we tried to solve is: what if the server simply could not read your files, even if it wanted to?

We can solve this problem through cryptography. But we may face difficulties while solving this issue. Three core challenges need to be addressed simultaneously.

- Confidentiality : Ensuring that file contents are unreadable to anyone who isn't supposed to see them.
- Integrity : Guaranteeing that a file hasn't been silently modified or corrupted in transit or at rest.
- Authentication : Making sure that only legitimate users can access the system and that keys are tied to verified identities.

Solving all three together is what makes secure file sharing a challenging task.

We designed and implemented a server-side hybrid-encrypted file sharing platform. Files are encrypted at the backend before being written to storage the server never persists plaintext to disk. Private keys are never stored in plaintext, and a full database breach yields only encrypted blobs that cannot be read without the user's password.To make the cryptographic choices explicit and comparable, we built the system as a multi-algorithm demo platform. Users can choose between AES-256-GCM and ChaCha20-Poly1305 for file encryption. For integrity hashing they can choose between SHA-256, SHA-512, and MD5. We deliberately left MD5 in as a broken option so the difference between a strong and a weak hash algorithm can be compared. And lastly, file sharing is handled through hybrid RSA encryption, where a file is encrypted once but its key can be securely handed to any number of recipients without ever exposing the key itself to the server.

This report walks through how we implemented this. The System Design section covers our architecture and the cryptographic decisions behind each component. The Implementation section describes the working prototype and how the major features were coded. The Security Analysis identifies the vulnerabilities we found in our own design and the mitigations we propose. Finally, the Conclusion reflects on what we built, what we learned, and what we would do differently with more time.

2. System Design

2.1 Scenario

The system we built is a web-based secure file sharing platform. The core idea is: a user should be able to upload a file, store it on a remote server, and share it with another user. But all this without the server ever being able to read the file's contents. This scenario was chosen because it maps directly onto a real and widespread problem. There are tools like Google Drive, Dropbox, and OneDrive that are convenient, but they all rely on server-side encryption. It means that the provider holds the keys and can technically access your data. This poses a risk for data breach. Our platform eliminates that trust requirement entirely.

The workflow is as follows. When a user registers, a unique RSA key pair is generated for them. When they upload a file, the file is encrypted on the backend before being written to the database. The server then stores only ciphertext and never sees the plaintext. A cryptographic hash of the original file is also computed before encryption and stored alongside it. When the file is downloaded and decrypted, this hash is recomputed and compared against the stored value to verify that the file has not been altered. When a user shares a file with someone else, they do not re-encrypt the file. Instead they re-wrap the file's encryption key using the recipient's public key, so only the recipient can unwrap it. The server facilitates all of this without ever persisting an unprotected key or plaintext file to disk. Plaintext exists only transiently in server memory during active upload and download requests.

2.2 Cryptographic techniques

The system uses a layered set of cryptographic primitives. Each chosen for a specific purpose. This section explains what each algorithm does, why it was selected, and what alternatives were considered and rejected.

AES-256-GCM : Symmetric File Encryption

AES-256-GCM is the primary algorithm used to encrypt file contents. AES (Advanced Encryption Standard) with a 256-bit key is a well-established symmetric cipher standardized by NIST and is the only publicly accessible cipher approved by the NSA for top-secret information [1]. The GCM (Galois/Counter Mode) mode of operation is what makes it particularly well-suited here: GCM is an Authenticated Encryption with Associated Data (AEAD) scheme, meaning it provides both confidentiality and integrity in a single operation [2]. When a file is encrypted, GCM automatically produces an authentication tag stored alongside the ciphertext. On

decryption, this tag is verified before any plaintext is returned. If the ciphertext has been tampered with, decryption fails immediately.

The main alternative was AES-CBC (Cipher Block Chaining). CBC provides confidentiality only. A separate HMAC must be computed and verified to achieve integrity, and implementing this correctly is notoriously error-prone. Incorrect CBC-HMAC implementations have historically led to padding oracle attacks. GCM avoids all of this by integrating authentication into the encryption itself [2].

RSA-OAEP : Asymmetric Key Wrapping

RSA is used for key wrapping, not for encrypting files directly. RSA-2048 with OAEP padding can only encrypt approximately 190 bytes of data. The correct and standard approach is called hybrid encryption. It is to encrypt the file with a fast symmetric key (the File Encryption Key, or FEK), and then encrypt only that key with RSA. A random 256-bit FEK is generated per file, used to encrypt the file with AES-GCM. The FEK itself is then wrapped with the owner's RSA public key and stored in the database.

ChaCha20-Poly1305 : Alternative Symmetric Encryption

ChaCha20-Poly1305 is offered as a second encryption option alongside AES-256-GCM. Like AES-GCM, it is an AEAD cipher. ChaCha20 handles the stream encryption while Poly1305 provides the authentication tag. It is standardized in RFC 8439 [3] and has been adopted in TLS 1.3, OpenSSH, and WireGuard. It is one of the most widely vetted modern ciphers available.

The practical motivation for including it is hardware independence. AES-GCM performs best on CPUs that support the AES-NI hardware instruction set. On devices without hardware AES acceleration, software AES can be significantly slower and more susceptible to timing side-channel attacks. ChaCha20 is a pure software cipher that performs consistently well regardless of hardware, with no timing side channels. Including both algorithms allows us to demonstrate and compare two production-grade symmetric encryption choices.

SHA-256 and SHA-512 : File Integrity Hashing

Before a file is encrypted and uploaded, a cryptographic hash of its plaintext content is computed and stored in the database. When the file is later downloaded and decrypted, the hash is recomputed and compared against the stored value to verify file integrity.

SHA-256 and SHA-512 are both members of the SHA-2 family, standardized by NIST in FIPS 180-4 [4]. Both are cryptographically secure: they are one-way and collision-resistant. It is computationally infeasible to find two different inputs that produce the same digest. SHA-256

produces a 256-bit (64 hex character) digest and is the most widely used general-purpose hash. SHA-512 produces a 512-bit (128 hex character) digest and offers a higher security margin for contexts where long-term resistance is a priority.

MD5 : A Broken Algorithm (Included for Comparison)

MD5 is included deliberately as a negative example. It is a legacy hash function now considered cryptographically broken. At the Crypto 2004 conference, researchers demonstrated practical collision attacks against MD5, along with MD4, HAVAL-128, and RIPEMD [5].

We include MD5 in our hash algorithm selector because having it side-by-side with SHA-256 and SHA-512 makes the comparison concrete and visible. A user can upload a file, see all three digests at once, and observe the difference in output length and security. The goal is to demonstrate how a broken algorithm behave identically to a secure one, until it doesn't

‘bcrypt’ : Password Hashing

User passwords are hashed using ‘bcrypt’. It is a purpose-built password hashing function introduced by Provos and Mazières in 1999 [6]. It is important to understand why password hashing requires a different approach than file hashing. SHA-256 and SHA-512 are designed to be fast. This is exactly what we want for file integrity, but is dangerous for passwords. An attacker with a database of SHA-256 password hashes can test billions of guesses per second, recovering common passwords almost instantly.

‘bcrypt’ is designed to be slow. It incorporates a configurable cost factor. We use a cost factor of 12, requiring approximately 300 milliseconds per hash on modern hardware. This makes brute-force attacks several orders of magnitude harder. ‘bcrypt’ also automatically generates and embeds a random salt into every hash, making precomputed rainbow tables useless. On login, the stored salt is extracted from the hash and used to re-derive the hash from the entered password for comparison.

The system derives a 256-bit AES key from the user's password using PBKDF2-HMAC-SHA256 with 100,000 iterations and a random salt. This derived key encrypts the RSA private key using AES-256-GCM before it is written to the database. Even if the database is fully compromised, an attacker still cannot use any private key without also cracking the corresponding user's password.

2.3 Server-side encryption model

A central design goal of this system is that the server should be structurally blind to all sensitive data. It is not just as a policy, but by architecture. The table below summarizes what is and is not accessible to the server at any point.

What the server STORES	What the server never persists to disk
Encrypted file ciphertext (AES-GCM or ChaCha20)	Plaintext file contents at rest (processed transiently in memory only)
Wrapped File Encryption Key (RSA-encrypted FEK)	The raw FEK (plaintext symmetric key)
RSA public key in PEM format	RSA private key in plaintext
AES-GCM encrypted RSA private key blob	User's password (only its ‘bcrypt’ hash is stored)
PBKDF2 salt and nonce for private key decryption	The AES key derived from the user's password
‘bcrypt’ hash of the user's password	Any key material in usable, unencrypted form
SHA-256/SHA-512/MD5 hash of the plaintext file	The decrypted file at rest (only in memory during download)

The practical implication of this model is that a full database breach yields no usable file contents and no usable private keys. Plaintext is only present in server memory during active requests and is never written to the database. All an attacker would have is encrypted blobs and public keys. To decrypt any file, they would still need to obtain the corresponding user's password and perform the PBKDF2 key derivation to unlock the private key, which in turn unlocks the FEK, which in turn decrypts the file. The system provides no shortcut around this chain.

2.4 Diagrams

Diagrams are compiled as PNGs from PUML file under docs/img/

Diagram	File	Caption
User Registration	us-1-register-user.png	Figure 1 User registration and RSA key generation flow
User Login	us-1-login-user.png	Figure 2 Login and JWT issuance flow
File Upload & Encryption	us-2-encryption.png	Figure 3 File encryption and upload sequence
File Listing & Delete	us-3-file-system.png	Figure 4 File listing sequence
File Delete	us-3-file-delete.png	Figure 5 File deletion with ownership check
File Download	us-4-file-download.png	Figure 6 File decryption and integrity verification sequence
File Sharing	us-6-share-file.png	Figure 7 FEK re-wrapping for recipient
Shared Download	us-6-download.png	Figure 8 Shared file download by recipient
File Edit	us-6-file-edit.png	Figure 9 Re-encryption and FEK re-wrap on edit

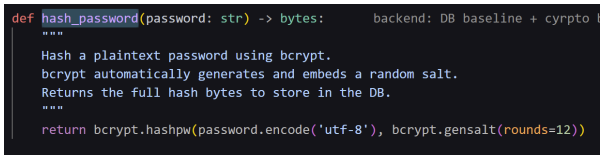
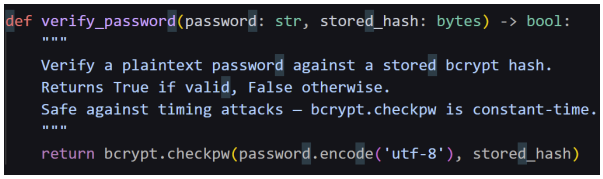
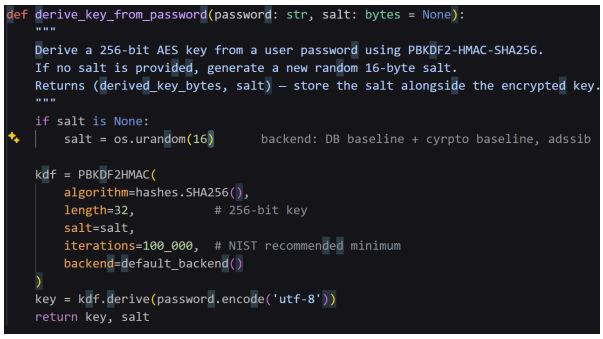
3. Implementation

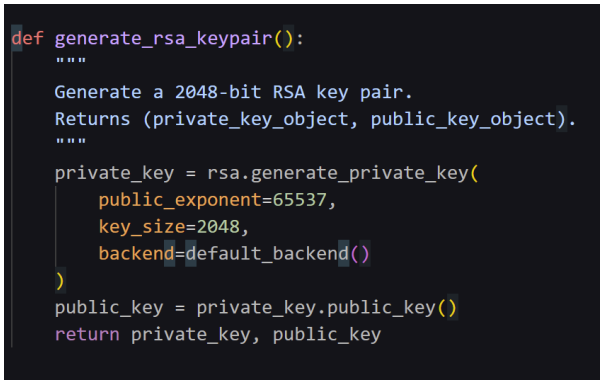
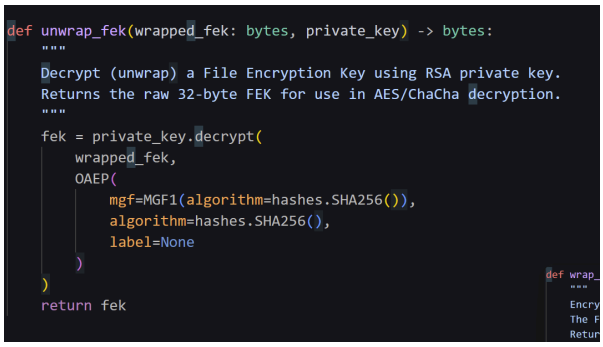
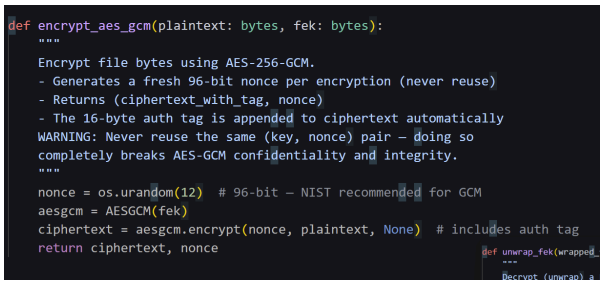
Github Repository: https://github.com/codedsami/SOEN-321-Project

3.1 Tech stack

The backend is implemented in Python 3.10 using Flask as the web framework, chosen for its minimal footprint and straightforward route registration, which keeps the codebase focused on cryptographic logic rather than framework boilerplate. All cryptographic operations are handled exclusively by PyCA's cryptography library a widely audited, low-level library that exposes direct access to primitives such as AES-GCM, ChaCha20-Poly1305, RSA-OAEP, and PBKDF2 without abstracting away the parameters that matter for security. No custom cryptographic code was written; every primitive is delegated to the library's hazmat layer. Authentication is managed by Flask-JWT-Extended, which handles JWT issuance and verification on protected routes. The database is SQLite via Flask-SQLAlchemy, sufficient for a prototype of this scope the schema stores only encrypted artifacts and never plaintext. The frontend is plain HTML, CSS, and JavaScript served directly by Flask, using Bootstrap 5 via CDN for layout and the browser's native fetch API for all backend calls, requiring no build toolchain or additional dependencies.

3.2 Core crypto functions

Function Name	Function code
hash_password	
verify_password	
derive_key_from_password	

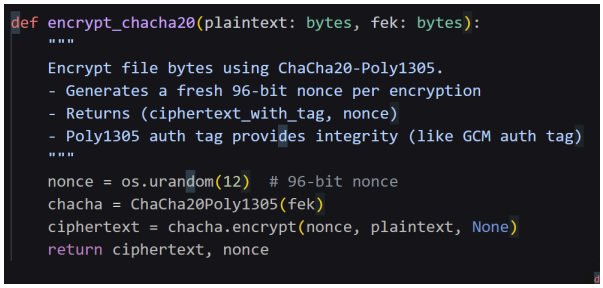
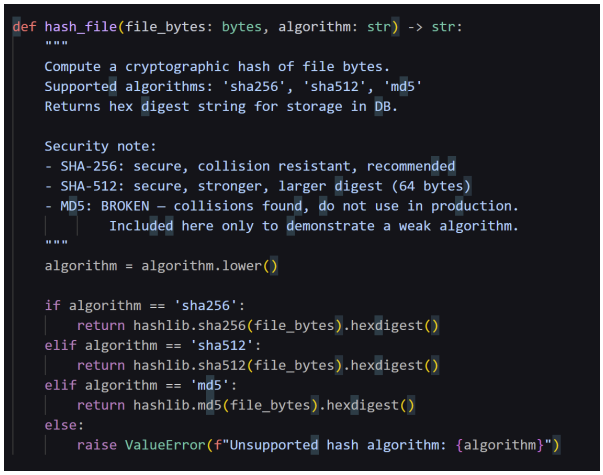
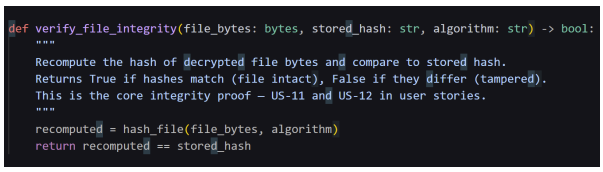
generate_rsa_keypair	
wrap_fek	
unwrap_fek	
encrypt_aes_gcm	

```python
def generate_rsa_keypair():
    """
    Generate a 2048-bit RSA key pair.
    Returns (private_key_object, public_key_object).
    """
    private_key = rsa.generate_private_key(
        public_exponent=65537,
        key_size=2048,
        backend=default_backend()
    )
    public_key = private_key.public_key()
    return private_key, public_key
```

```python
def wrap_fek(fek: bytes, public_key) -> bytes:
    """
    Encrypt (wrap) a File Encryption Key using RSA-OAEP.
    The FEK is a 32-byte AES key — well within RSA-2048 limits.
    Returns the wrapped FEK bytes to store in the DB.
    """
    wrapped = public_key.encrypt(
        fek,
        OAEP(
            mgf=MGF1(algorithm=hashes.SHA256()),
            algorithm=hashes.SHA256(),
            label=None
        )
    )
    return wrapped
```

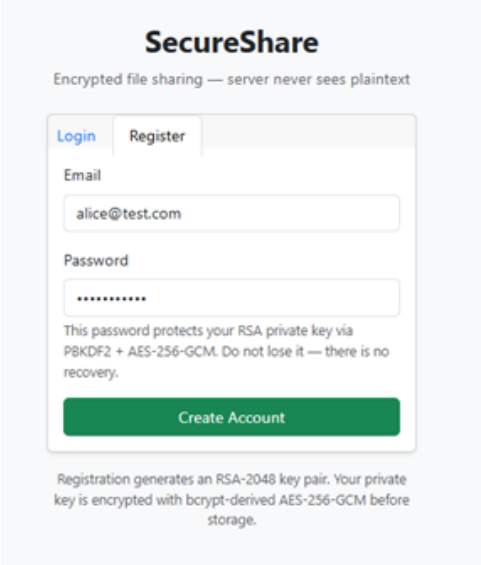
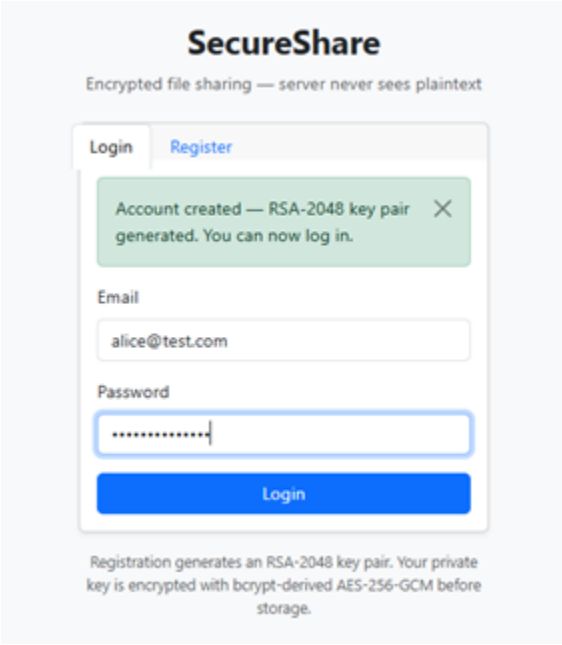
```python
def unwrap_fek(wrapped_fek: bytes, private_key) -> bytes:
    """
    Decrypt (unwrap) a File Encryption Key using RSA private key.
    Returns the raw 32-byte FEK for use in AES/ChaCha decryption.
    """
    fek = private_key.decrypt(
        wrapped_fek,
        OAEP(
            mgf=MGF1(algorithm=hashes.SHA256()),
            algorithm=hashes.SHA256(),
            label=None
        )
    )
    return fek
```

```python
def encrypt_aes_gcm(plaintext: bytes, fek: bytes):
    """
    Encrypt file bytes using AES-256-GCM.
    - Generates a fresh 96-bit nonce per encryption (never reuse)
    - Returns (ciphertext_with_tag, nonce)
    - The 16-byte auth tag is appended to ciphertext automatically
    WARNING: Never reuse the same (key, nonce) pair — doing so
    completely breaks AES-GCM confidentiality and integrity.
    """
    nonce = os.urandom(12)  # 96-bit — NIST recommended for GCM
    aesgcm = AESGCM(fek)
    ciphertext = aesgcm.encrypt(nonce, plaintext, None)  # includes auth tag
    return ciphertext, nonce
```

encrypt_chacha20	
hash_file	
verify_file_integrity	

3.3 Sample output

Full walkthrough: [Walkthrough - click here]

Feature & Description	Screenshot
User Registration First the user registers an account in the database Registration creates a unique RSA-2048 key pair for the user. The private key is immediately encrypted using AES-256-GCM with a key derived from the user's password via PBKDF2 before being stored in the database. The server never holds the private key in plaintext.	
Login (confirmation of RSA key pair generation)	

File Upload & Encryption

When a file is uploaded, the system:

1. Computes a cryptographic hash of the plaintext file before encryption

2. Generates a fresh random 256-bit File Encryption Key (FEK) via os.urandom

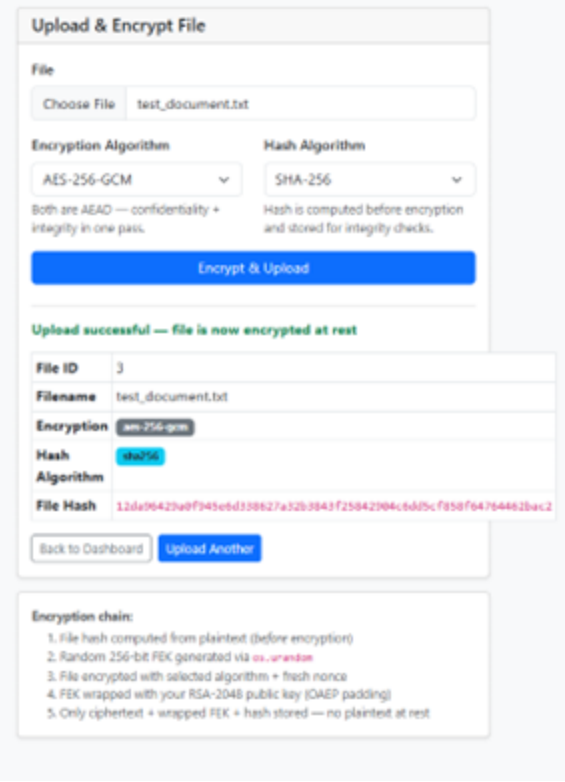
3. Encrypts the file using the selected algorithm and a fresh nonce

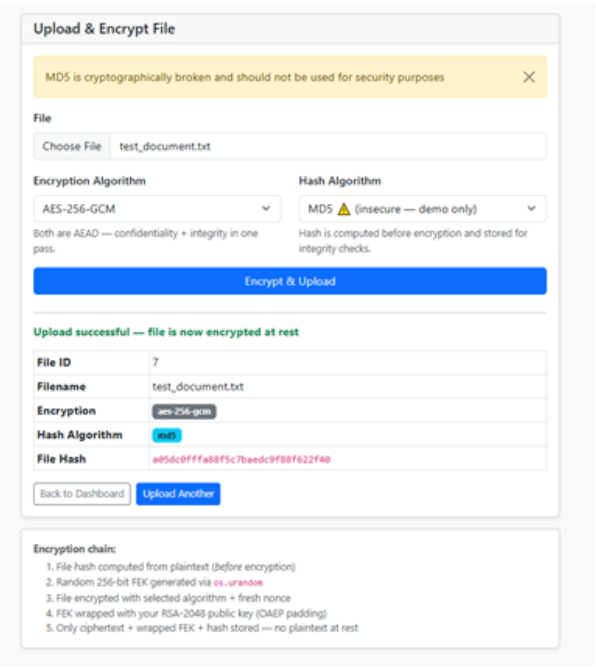
4. Wraps the FEK with the user's RSA-2048 public key (OAEP padding)

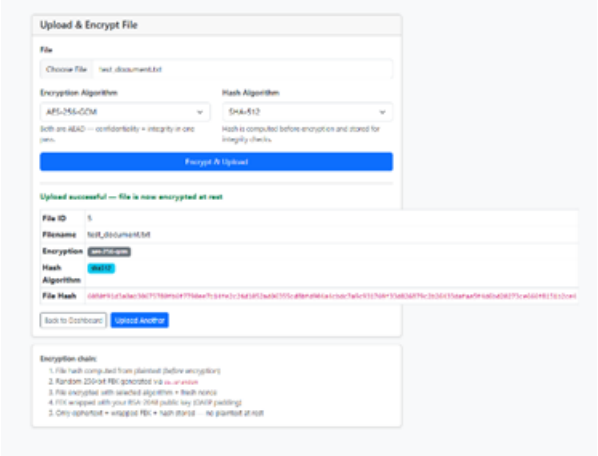
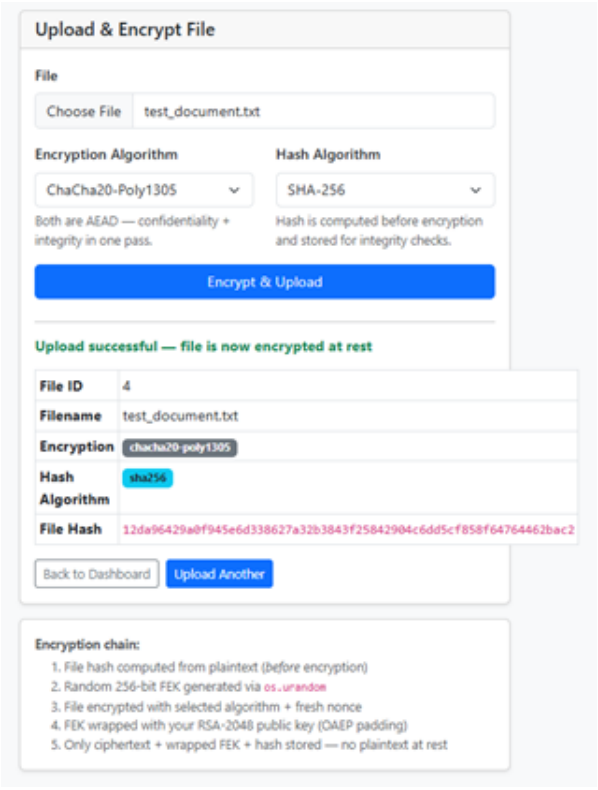
5. Stores only the ciphertext, wrapped FEK, nonce, and hash — never the plaintext

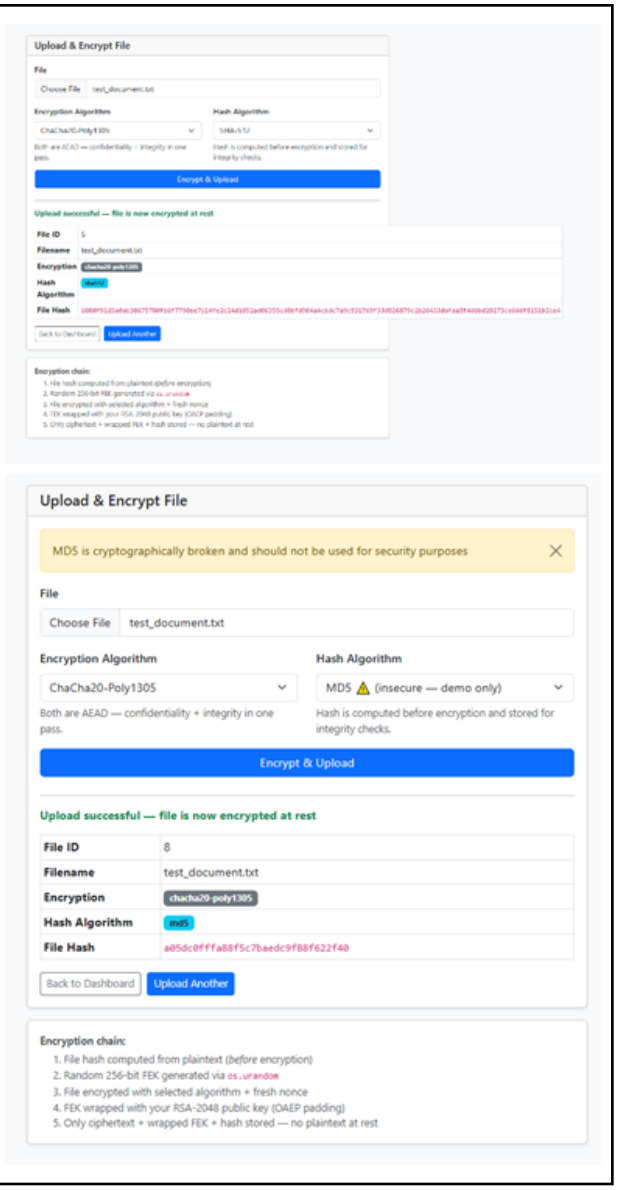
Encryption Algorithm: AES-256-GCM

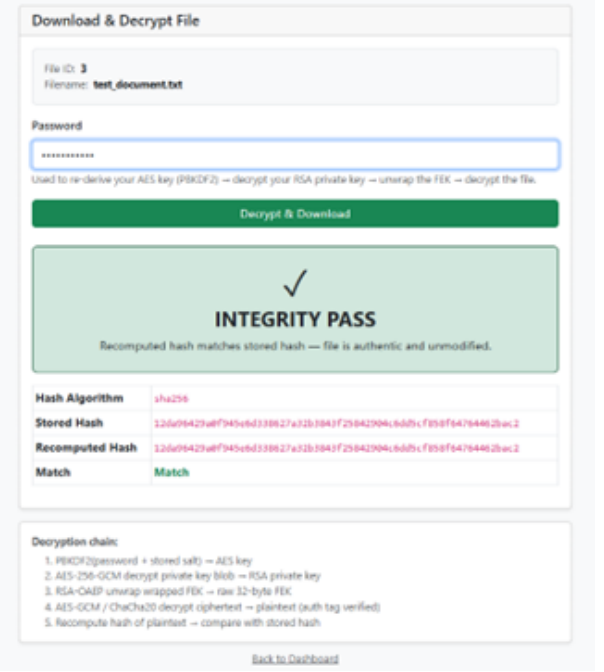
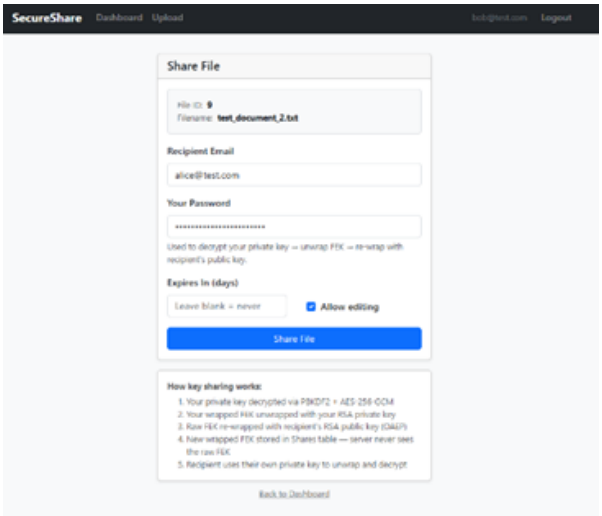
Hash Algorithm: SHA-256, SHA-512 or MD5





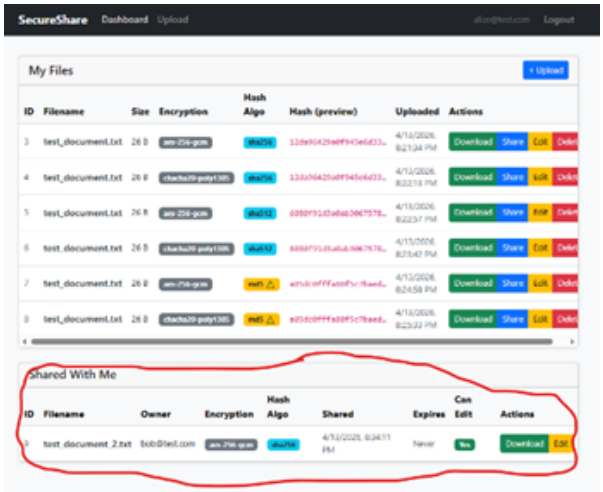
	
File Upload & Encryption Encryption Algorithm: ChaCha20-Poly1305 Hash Algorithm: SHA-256, SHA-512 or MD5	

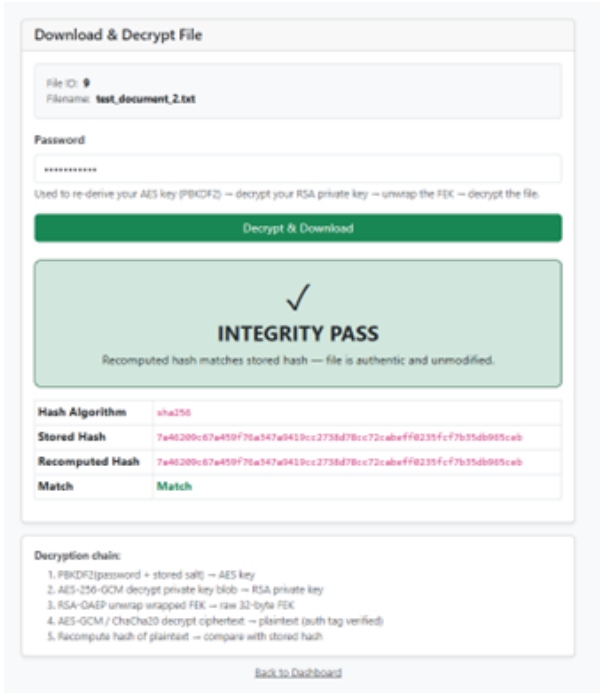

Upload & Encrypt File
MD5 is cryptographically broken and should not be used for security purposes
File
Choose File
test_document.txt
Encryption Algorithm
ChaCha20-Poly1305
Both are AEAD — confidentiality + integrity in one pass.
Hash Algorithm
MD5 ⚠ (insecure — demo only)
Hash is computed before encryption and stored for integrity checks.
Encrypt & Upload
Upload successful — file is now encrypted at rest
File ID
8
Filename
test_document.txt
Encryption
chacha20-poly1305
Hash Algorithm
md5
File Hash
a05dc0fffa88f5c7baedc9f88f622f40
Back to Dashboard
Upload Another
Encryption chain:
1. File hash computed from plaintext (before encryption)
2. Random 256-bit FEK generated via os.urandom
3. File encrypted with selected algorithm + fresh nonce
4. FEK wrapped with your RSA-2048 public key (OAEP padding)
5. Only ciphertext + wrapped FEK + hash stored — no plaintext at rest

Downloading Securely — Integrity Check PASS To decrypt a file, the user provides their login password. The system runs the full decryption chain: 1. PBKDF2(password + stored salt) → AES key 2. AES-256-GCM decrypts the private key blob → RSA private key 3. RSA-OAEP unwraps the wrapped FEK → raw 32-byte FEK 4. AES-GCM / ChaCha20 decrypts the ciphertext (authentication tag verified) 5. Hash of the decrypted plaintext is recomputed and compared to the stored hash	
Sharing with different users We can see bob@test.com sharing a file with alice@test.com Alice receiving the file and being able to securely download it using her login password The share operation: 1. Decrypts Bob's private key using his password 2. Unwraps the FEK using Bob's private key 3. Re-wraps the same FEK using Alice's RSA	

public key (fetched from DB)

4. Stores a new Share record containing Alice's wrapped FEK





Editing Shared Files

U1 that received the shared file has decided to edit it, this process replaces the current file and Re-encrypts it.

Then if we have a look into U2's uploaded file we can see that the shared file has the edited files name as well as the edited files content.

Test_document_2.txt->"This is the test document number 2!"

Test_document_2-1.txt->"This is the test document number 2-1!"

When a file is edited, the system performs a full cryptographic reset:

1. Hashes the new plaintext before encryption

2. Generates a brand-new FEK — the old FEK is discarded

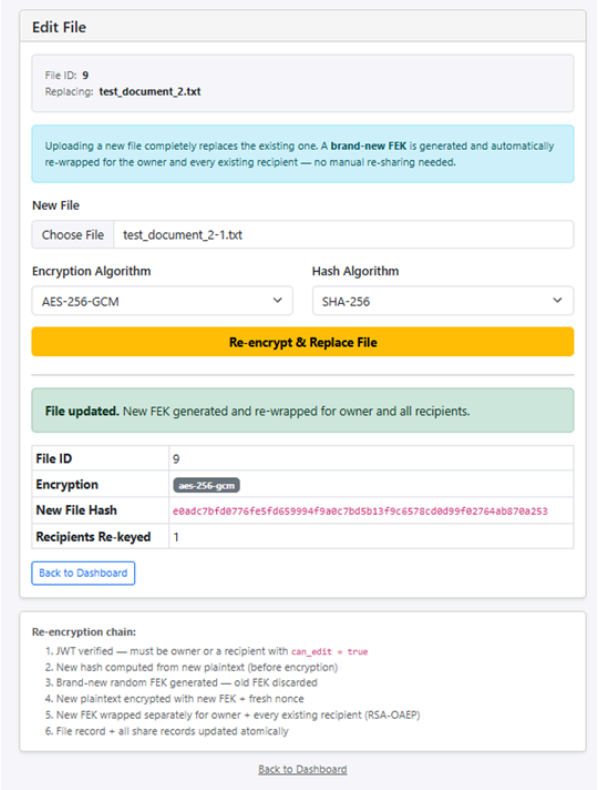
3. Encrypts the new content with the new FEK and a fresh nonce

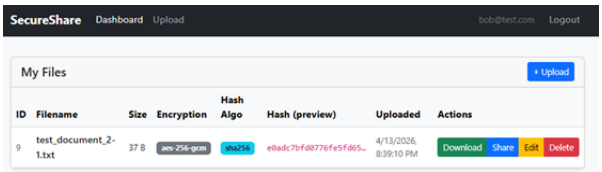
4. Re-wraps the new FEK for the owner using the owner's public key

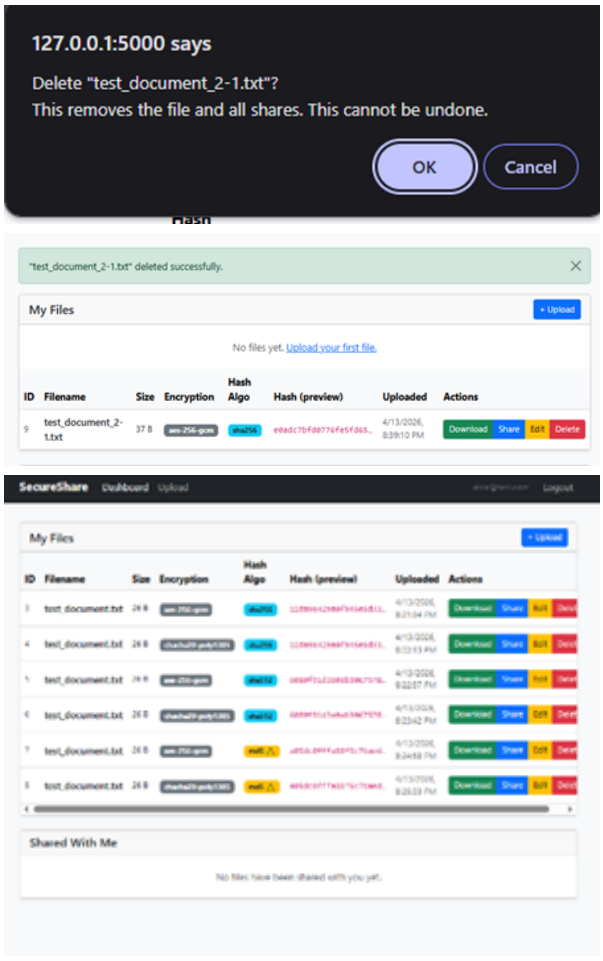
5. Loops through every existing Share record and re-wraps the new FEK for each recipient using their public key

6. Updates the File record and all Share records atomically

No manual re-sharing is required.





Recipients automatically receive access to the updated file through their re-wrapped FEK.	
Deleting a File Only the file owner can delete a file. Deletion removes the ciphertext, nonce, wrapped FEK, all metadata, and all Share records in a single cascaded operation. Once deleted, no recipient retains access.	

4. Security Analysis and Discussion

4.1 Threat model

This system is designed against three primary adversaries. They consist of a passive eavesdropper, a compromised server, and a malicious insider. First, the passive eavesdropper can observe network traffic and attempt to infer file contents, keys, or other sensitive information in transit. Next, the compromised server adversary can fully access backend application logic, API responses, and database contents. Finally, the malicious insider has legitimate privileged access to server-side infrastructure and may intentionally misuse it. These threats are particularly relevant in cloud storage systems, where the provider's infrastructure is often the main point of exposure. Our design reduces this risk by ensuring that the backend stores only encrypted artifacts, while never storing plaintext files, raw symmetric keys, plaintext private keys, or user passwords. The encrypted artifacts include ciphertext, RSA-wrapped file-encryption keys, bcrypt password hashes, and AES-encrypted private-key blobs. Therefore, a server-side compromise should reveal metadata and encrypted material, but not directly reveal user plaintext. The model assumes honest cryptographic primitives, an uncompromised client environment, and that server memory is not accessible to an attacker between requests. In other words, it does not attempt to defend against malware running on the user's own device.

4.2 Vulnerabilities and mitigations

Nonce reuse in AES-GCM

AES-GCM provides confidentiality and integrity, but it becomes unsafe if the same nonce is reused with the same key. In our system, this risk could appear during repeated uploads, retries, or file edits if an old FEK and nonce were accidentally reused. This could leak relationships between ciphertexts or weaken authentication. We mitigate this by generating a fresh random FEK and a fresh nonce for every encryption or re-encryption operation. This stores the nonce only as metadata for later decryption. Moreover, it uses a cryptographically secure source such as os.urandom for all nonce generation [2][3].

MD5 collision vulnerability

MD5 is no longer secure for adversarial integrity checking because attackers can construct different inputs with the same digest. Since our system verifies file integrity by hashing plaintext before upload and comparing the recomputed hash after download, choosing MD5 could allow a tampered file to appear valid even when its content has changed. The mitigation is to treat MD5 as a demonstration-only option and explicitly mark it as insecure. It would also be to use SHA-256 or SHA-512 as the real integrity choices in practice [4][5].

RSA without OAEP padding

RSA encryption is only secure when used with proper padding. Indeed, weaker padding can expose the system to chosen-ciphertext or padding-oracle style attacks. In our design, RSA is used only to wrap the FEK. This means that any weakness in RSA wrapping would undermine

access control over the encrypted file itself. We mitigate this by using hybrid encryption correctly. That is, the file is encrypted with a symmetric FEK, and only that FEK is wrapped using RSA-OAEP, with generic failure handling and well-tested library code rather than custom RSA logic.

Weak password hashing

Hash functions such as SHA-256 are suitable for file integrity but unsafe for password storage because they allow large-scale offline guessing attacks. This matters greatly in our system because the user's password protects both account access and the AES key derived through PBKDF2 that encrypts the RSA private key. If password hashes were weak, an attacker could potentially recover private keys and then decrypt FEKs and files. The mitigation is to store passwords with bcrypt using a unique salt and suitable work factor. In parallel, we reserve SHA-256 and SHA-512 only for file hashing [4][6].

Server compromise

A compromised server is one of the most serious threats in cloud storage because traditional systems often store both encrypted files and the means to decrypt them. Our design reduces this risk by ensuring that the server stores only ciphertext, wrapped FEKs, encrypted private-key blobs, public keys, and password hashes. It also never stores plaintext files, raw FEKs, plaintext private keys, or user passwords. The mitigation is therefore architectural. Encrypted key storage and server-side hybrid encryption ensure that a database breach exposes only encrypted artifacts and metadata, not immediately readable file contents.

Man-in-the-middle on key exchange

A man-in-the-middle attacker can break confidentiality by substituting a public key during transmission. This would cause the client to encrypt the FEK to the attacker instead of the intended user. This risk applies to our system whenever the client fetches a public key from the server during upload or file sharing, because zero-knowledge storage alone does not prove that the returned public key is authentic. The mitigation is to use TLS/HTTPS for transport security and authenticate public keys through fingerprints, certificates, or another trusted key-distribution mechanism.

The most significant architectural limitation is that all cryptographic operations are performed server-side rather than in the client's browser. This means the server processes plaintext in memory during every upload and download request. The originally intended design called for client-side cryptography using the Web Crypto API, which would have made plaintext invisible to the server entirely. This was descoped due to the absence of a browser-based frontend in early development. A production version should move all encryption, decryption, and key derivation to the client.

Weak random number generation

Cryptographic security depends on unpredictable randomness for FEKs, nonces, and salts. If these values are generated with a predictable pseudo-random source, attackers may guess or reproduce them. In our system, weak randomness would directly threaten file encryption, AEAD nonce uniqueness, PBKDF2 salting, and private-key protection. The mitigation is to generate all

security-sensitive values with a cryptographically secure operating-system source such as os.urandom. Another mitigation is also to centralize randomness generation in dedicated helper functions to prevent accidental misuse.

4.3 Limitations of the prototype

Considering all of the aforementioned, the prototype should not be viewed as a production-ready secure file-sharing system. Despite demonstrating the main ideas behind zero-knowledge storage, hybrid encryption, password-protected private keys, and integrity verification, it still has important limitations. Specifically, the current design does not provide digital signatures, forward secrecy, or a trusted public-key infrastructure.

First, the system does not use digital signatures. As such, it can verify file integrity but not strong authorship or non-repudiation. A downloaded file can be checked against its stored hash, but that only shows consistency with the stored record. It does not prove who created the file or who last modified it. It also does not reveal whether a shared file truly came from the claimed sender. Furthermore, the system lacks certificate authority or other trusted public-key infrastructure. This means it assumes that the public keys returned by the server are correct. In a real system it leaves room for public-key substitution attacks if the server is compromised or malicious. A better design would bind identities to keys using certificates or trusted fingerprints.

Second, the prototype does not provide forward secrecy. Access to each file ultimately depends on long-term RSA private keys that are themselves protected by password-derived encryption. As a result, if an attacker later obtains a user's password and decrypts the stored private key, previously wrapped FEKs may also become recoverable. This puts past encrypted files at risk. In a system with forward secrecy, compromise of long-term credentials should not automatically expose previously protected data. A more advanced version of the system could address this by introducing ephemeral key exchange mechanisms such as ECDH-based designs.

Finally, the prototype focuses on demonstrating cryptographic workflows rather than full operational hardening. Metadata such as filenames, file size, upload dates, owners, recipients, and selected algorithms is still visible to the server. Therefore, the system is zero-knowledge with respect to file contents, not total secrecy. In addition, security still depends heavily on the client environment. That is to say that if the user's browser, device, or session is compromised, passwords and decrypted plaintext risk also to be exposed.

All these limitations are natural boundaries of a first prototype rather than failures of the design. The system successfully demonstrates how cryptography can reduce trust in the server. However, it does not yet provide stronger authenticity, key-distribution assurance, and long-term protection expected from a mature secure communication platform. Thus, future enhancements should prioritize digital signatures, authenticated public-key infrastructure, and forward-secret key exchange.

5. Conclusion

This project set out to answer a practical security question: how do you build a file sharing system where a compromised server cannot read user data? The result is a working prototype that demonstrates the full cryptographic chain required to get there hybrid encryption with AES-256-GCM and ChaCha20-Poly1305 for file confidentiality, RSA-2048 OAEP for key wrapping, PBKDF2 for password-based key derivation, bcrypt for password storage, and hash-based integrity verification across all supported algorithms.

The most important takeaway from the security analysis is that algorithm choice and architectural placement matter as much as each other. Choosing AES-256-GCM over AES-CBC eliminates an entire class of padding oracle vulnerabilities. Choosing bcrypt over SHA-256 for passwords makes brute-force attacks orders of magnitude harder. But the biggest security property in this system that a database breach yields nothing usable comes not from any single algorithm, but from the overall design: every sensitive artifact stored at rest is encrypted, and the keys to decrypt them are themselves encrypted under keys that only the user can derive.

The prototype has clear limitations. The most significant is that all cryptographic operations run server-side, meaning the server processes plaintext in memory during each request. A production-grade version would move all encryption and decryption to the browser using the Web Crypto API, making plaintext structurally inaccessible to the server. Beyond that, the system lacks digital signatures for non-repudiation, a trusted public-key infrastructure to prevent key substitution attacks, and forward secrecy meaning a compromised password could expose all past files. Future iterations should address these gaps by introducing ECDH-based ephemeral key exchange, certificate-bound identities, and client-side cryptography as the foundation rather than an afterthought.

6. References

1. National Institute of Standards and Technology. (2001). Advanced Encryption Standard (AES). Federal Information Processing Standard 197. https://csrc.nist.gov/pubs/fips/197/final
2. National Institute of Standards and Technology. (2007). Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC. NIST Special Publication 800-38D. https://csrc.nist.gov/pubs/sp/800/38/d/final
3. Nir, Y., & Langley, A. (2018). ChaCha20 and Poly1305 for IETF Protocols. RFC 8439. Internet Engineering Task Force. https://datatracker.ietf.org/doc/rfc8439/
4. National Institute of Standards and Technology. (2015). Secure Hash Standard (SHS). Federal Information Processing Standard 180-4. https://csrc.nist.gov/pubs/fips/180-4/upd1/final
5. National Institute of Standards and Technology. (2004). NIST Brief Comments on Recent Cryptanalytic Attacks on Secure Hash Functions. CSRC. https://csrc.nist.gov/News/2004/NIST-Brief-Comments-on-Recent-Cryptanalytic-Attack
6. Provos, N., & Mazières, D. (1999). A future-adaptable password scheme. In Proceedings of the 1999 USENIX Annual Technical Conference, pp. 81–91. USENIX Association.